

Rapport final de projet

Honeypot multi-services (SSH / HTTP / FTP), pipeline d'analyse en temps reel et generation de renseignement sur les menaces (CTI).

Auteurs	Mohammed Sellak
	Rachid Oubenazha
Formation	Master 1 – Securite des Systemes (M1SPRO)
Etablissement	Ecole IT d'Amiens
Module	Projet Honeypot – 35 heures
Date	12 juin 2026

Resume executif

Ce projet realise un **honeypot intelligent multi-services** conçu pour attirer, capturer et analyser les attaques visant des services exposes. Au-dela de la simple capture, le systeme constitue une **chaîne complete de renseignement sur les menaces** : chaque interaction est journalisee de maniere integre (signature HMAC-SHA256), enrichie (geolocalisation, reputation), classee selon un profil comportemental, visualisee en temps reel sur un tableau de bord de type SOC, puis exportee vers des formats defensifs standards (Sigma, STIX 2.1, listes iptables).

Lors de la campagne de validation, le systeme a capture **5 899 evenements** sur les trois services, avec **0 log corrompu** (integrite verifiee de bout en bout), et a classe automatiquement les attaquants en quatre profils. L'audit de detectabilite montre une progression de la furtivite de **6/30 a 21/30** apres durcissement.

Sommaire

1. Introduction et contexte

Tout service expose sur un reseau subit un bruit d'attaque permanent : balayages automatisés, attaques par force brute, tentatives d'exploitation de vulnérabilités. Distinguer une activité réellement malveillante d'un balayage de routine est difficile, et la collecte de renseignement exploitable reste un enjeu central pour toute équipe défensive (Blue Team).

1.1 Objectifs du projet

Le projet poursuit deux objectifs complémentaires :

1. **Detourner** une partie du trafic offensif vers des leurres sans valeur, afin d'observer les techniques adverses sans exposer de ressource réelle ;
2. **Transformer** les interactions capturées en renseignement structuré et directement exploitable (visualisation temps réel + exports CTI).

1.2 Perimetre et contraintes

Le système couvre trois protocoles représentatifs (SSH, HTTP, FTP), s'exécute entièrement en conteneurs et doit rester **passif**, **sécurisé** et **conforme** au cadre légal (RGPD, article 323-1 du Code pénal).

2. Cadre théorique et choix de conception

2.1 Taxonomie de Spitzner

Selon la taxonomie de L. Spitzner, on distingue trois niveaux d'interaction. Le tableau ci-dessous résume le compromis associé à chacun.

Niveau	Données recueillies	Risque
Faible	Connexions, bannières	Tres faible
Moyen (choisi)	Identifiants, commandes, payloads	Maîtrise
Fort	Comportement post-exploitation complet	Élevé (pivot)

TABLE 1 – Compromis richesse / risque des niveaux d'interaction.

2.2 Justification du medium-interaction

Nous retenons le niveau **moyen**. Il offre le meilleur ratio richesse/risque : un faux shell et des réponses applicatives crédibles donnent l'illusion d'un système réel *sans jamais exposer un système d'exploitation exploitable*. Cette approche s'aligne sur le framework **MITRE Engage** (objectifs *Collect, Detect, Direct, Reassure*).

3. Architecture du système

L'architecture suit un flux unique, de l'attaquant jusqu'au renseignement défensif. La figure ?? en présente la vue d'ensemble.

3.1 Choix techniques

4. Les services leurres

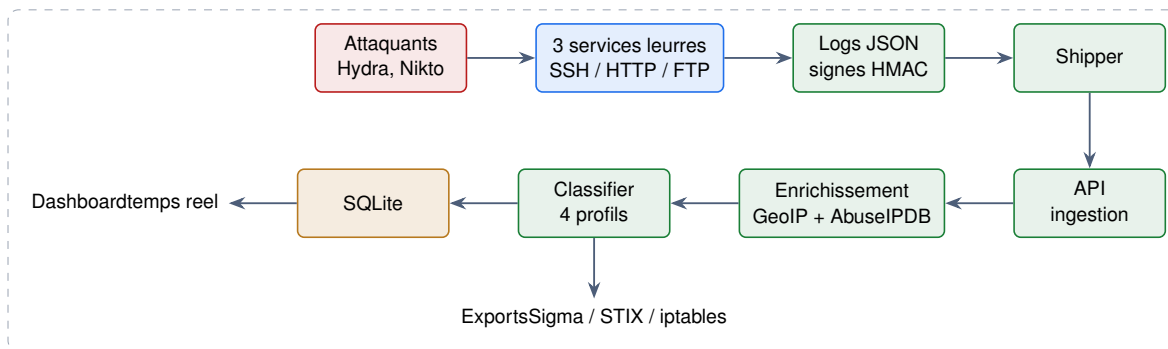


FIGURE 1 – Architecture de bout en bout : de l’attaque au renseignement défensif.

primary!8Composant	Technologie	Justification
SSH	Python asyncssh	Contrôle total du handshake et du faux shell
HTTP	Python FastAPI	Routage simple, middleware d’en-têtes
FTP	Python pyftplib	Serveur FTP scriptable, hooks d’événements
Ingestion	FastAPI + SQLite	Léger, sans serveur, suffisant au volume
Tableau de bord	Dash / Plotly	Visualisation temps réel en Python
Orchestration	Docker Compose	Reproductibilité, isolation réseau
Intégrité	HMAC-SHA256	Valeur probante des journaux (M1Cryp)

TABLE 2 – Pile technique retenue et justifications.

4.1 Service SSH (port 2222)

Le service capture chaque couple identifiant/mot de passe. Les identifiants faibles sont acceptés, ouvrant un **faux shell** (hostname `srv-prod-debian`) qui répond de manière cohérente aux commandes courantes (`uname -a`, `id`, `ps aux`, `netstat`, `cat /etc/passwd`, `history`...). La bannière est alignée sur une Debian 12 réelle : `SSH-2.0-OpenSSH_9.2p1 Debian-2+deb12u3`.

4.2 Service HTTP (port 8080)

Un serveur FastAPI expose des routes délibérément appatantes : `/.env`, `/.git/config`, `/phpinfo.php`, `/wp-login.php`, `/admin`, `/api/v1/users`. Un middleware réécrit les en-têtes pour simuler Apache/2.4.57 (Debian) avec `X-Powered-By: PHP/8.1.2`, la pile Python étant masquée (`server_header=False`).

4.3 Service FTP (port 2121)

Le serveur annonce 220 (`vsFTPD 3.0.5`) et propose un faux système de fichiers appatant (`secrets.txt`, `db_dump.sql`, `backup.zip`). Les actions `USER`, `PASS`, `LIST` et `RETR` sont intégralement journalisées.

4.4 Intégrité des preuves

Chaque événement est signé en **HMAC-SHA256**. L’ingestion recalcule la signature ; toute altération incrémente le compteur `integrity_failures`. Cette mesure garantit la valeur probante des journaux, indispensable à un usage forensique.

5. Pipeline d’analyse et classification

5.1 Contrat de log versionne

Les composants communiquent via un schema JSON versionne (schema_version 1.0.0), valide a l'ingestion (additionalProperties:false).

```

1 {
2   "schema_version": "1.0.0",
3   "event_id": "a1b2c3...",
4   "timestamp": "2026-06-11T15:12:25Z",
5   "service": "ssh",
6   "src_ip": "172.18.0.1",
7   "session_id": "e7f8...",
8   "action": "login_attempt",
9   "username": "root",
10  "password": "123456",
11  "integrity": "hmac-sha256:9f8a7b..."
12 }
```

Listing 1 – Exemple d'événement signé.

5.2 Enrichissement

Chaque adresse IP source est enrichie par geolocalisation (base GeoLite2-City) et, si une cle est fournie, par le score de reputation **AbuseIPDB**. Le mode degrade (hors-ligne) est gere proprement : absence d'enrichissement plutot qu'échec du pipeline.

5.3 Classification comportementale

Le classifieur agrege les evenements par session et applique une heuristique **explicable**, dont les seuils sont externalises dans `profiles.json`.

primary!8Profil	Heuristique principale	MITRE ATT&CK
bruteforcer	≥ 8 tentatives d'authentification, cadence elevee	T1110 Brute Force
bot	User-Agent automatise OU chemins / payloads d'exploit	T1595, T1190
human	Commandes shell variees, rythme humain	T1059, T1083
scanner_legitimate	UA Shodan/Censys/GreyNoise, faible volume	T1595

TABLE 3 – Les quatre profils comportementaux et leur correspondance ATT&CK.

Le classifieur retourne un objet {profil, confiance, raisons, mitre}, ce qui rend chaque decision **auditable** : il ne s'agit pas d'une boite noire.

6. Resultats et tableau de bord

Le tableau de bord (figure ??) restitue l'état du systeme en temps reel (rafraichissement toutes les 3 secondes) : indicateurs cles, services surveilles, carte geographique, repartition des profils, volumetrie par service et identifiants les plus testes.

6.1 Lecture des resultats

La campagne a produit les chiffres suivants, directement lisibles sur le tableau de bord :

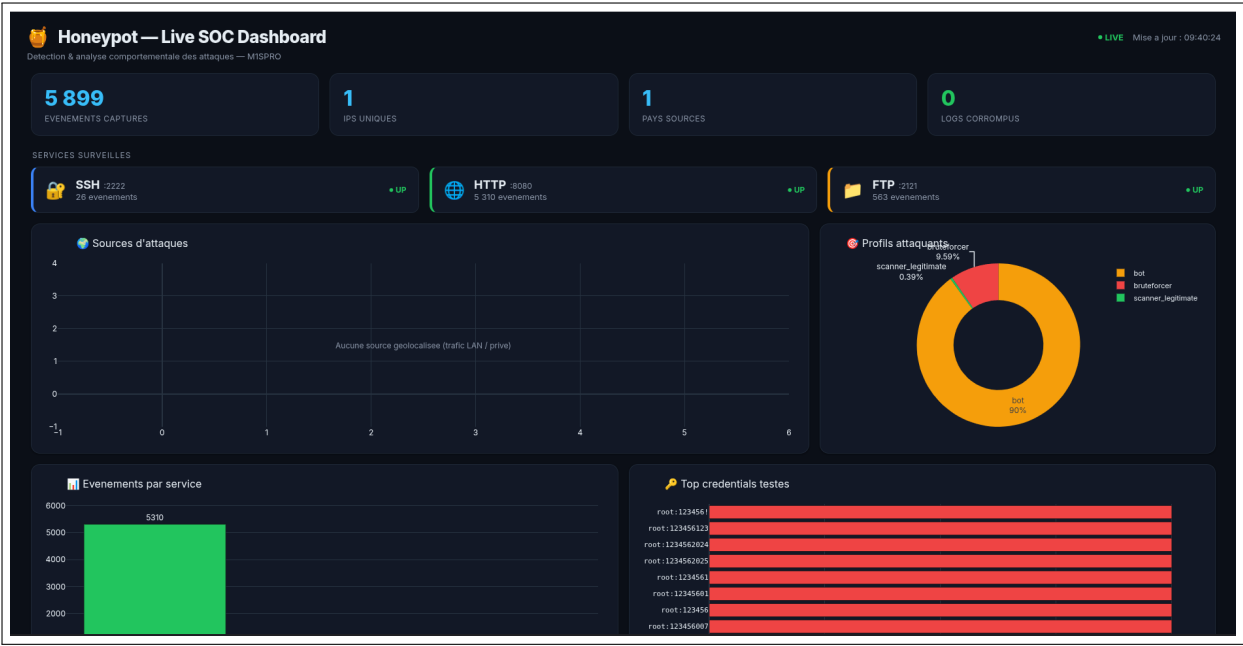


FIGURE 2 – Tableau de bord temps réel (« Live SOC Dashboard ») durant la campagne de validation.

primary !8Indicateur	Valeur	Service	Evenements
Evenements captures	5 899	SSH (:2222)	26
IPs uniques	1	HTTP (:8080)	5 310
Pays sources	1	FTP (:2121)	563
Logs corrompus	0	Total	5 899

TABLE 4 – Indicateurs globaux et volumetrie par service.

6.2 Interpretation

- **Domination du HTTP (5 310 evenements).** Le scan Nikto genere un tres grand nombre de requetes : c’est attendu et coherent avec un balayage de vulnerabilites. Ces sessions sont logiquement classees **bot** (90 % des profils).
- **Force brute SSH.** Les tentatives Hydra sur SSH alimentent le profil **bruteforcer** (9,59 %) ; les identifiants les plus testes sont des variantes de root : 123456, typiques des dictionnaires d’attaque.
- **Scanners legitimes (0,39 %).** Quelques sessions a faible volume avec un User-Agent connu sont correctement isolees, evitant les faux positifs.
- **Integrite parfaite (0 corrompu).** La verification HMAC valide la chaine de bout en bout : aucune preuve n’a ete alteree.
- **Carte vide « trafic LAN/privé ».** En environnement de test, le trafic provient d’une machine attaquante locale : l’adresse n’est pas geolocalisable. En exposition reelle, la carte se remplirait avec les pays sources. Cette limite est **assumee** et documentee.

7. Validation par attaques reelles

Des scripts reproductibles rejouent des attaques representatives :

- **Hydra** : force brute SSH et FTP (listes rockyou-top1000) ;
- **Nikto** et requetes cibles : scan de vulnerabilites HTTP ;

— payloads d'exploitation (/ .env, marqueurs Log4Shell).

Un script verifie automatiquement le bon fonctionnement de la chaine :

```
1 [PASS] >= 50 evenements captures (5899)
2 [PASS] service ssh present
3 [PASS] service http present
4 [PASS] profil bruteforcer detecte
5 [PASS] integrite des logs OK (0 corrompu)
6 -> 5/5 assertions reussies
```

Listing 2 – Sortie de attacks/assertions.py.

Resultat : l'ensemble de la chaine (capture, transport, ingestion, enrichissement, classification, persistance, restitution) est valide par 5 assertions sur 5, sur un volume reel de pres de 6000 evenements.

8. Audit de detectabilite (furtivite)

Un honeypot n'a de valeur que s'il n'est pas trivialement identifie comme tel. Nous avons mene un **audit avant/apres** avec les memes outils : nmap -sV + scripts NSE, p0f, Honeyscore (Shodan) et mesure de latence. La grille compte 10 criteres notes de 0 (detectable) a 3 (credible), soit un total sur 30.

prim	Crite#	Ini.	Fin.	Δ	Note finale
1	Banniere SSH (OpenSSH/Debian)	0	3	+3	Identique a Debian 12
2	Banniere HTTP (Server / X-Powered-By)	1	3	+2	Apache + PHP
3	Banniere FTP (vsFTPD)	1	3	+2	vsFTPD 3.0.5
4	Coherence uname / OS revendique	0	2	+2	bookworm coherent
5	Faux filesystem credible	0	2	+2	Fichiers cles presents
6	Latence / jitter des reponses	1	2	+1	Jitter 50–300 ms
7	Resistance NSE ssh-*	0	2	+2	Shell credible
8	Resistance NSE http-* / fingerprint	1	2	+1	En-tetes partiels
9	Empreinte pile TCP/IP (p0f)	1	1	0	Limite : OS hote visible
10	Honeyscore Shodan	1	1	0	N/A hors exposition
ok!	TOTAL	6	21	+15	

TABLE 5 – Score de furtivite : 6/30 → 21/30 (+15 points).

8.1 Contre-mesures appliquees

- **Bannières plausibles** : OpenSSH / Apache / vsFTPD alignees sur Debian 12;
- **Faux filesystem riche** : /etc/passwd, /etc/os-release, .bash_history, leurres FTP;
- **Reponses contextuelles** : faux shell coherent + jitter aleatoire.

Limites assumees. Deux criteres ne progressent pas, et c'est **structurel** : p0f observe la pile TCP/IP du noyau *hote* (non corrigeable cote applicatif ; mitigation en production : tuning sysctl TTL/MSS), et Honeyscore requiert une exposition publique, hors du perimetre LAN du projet.

9. Securite du honeypot et cadre legal

9.1 Durcissement (anti-pivot)

Le honeypot ne doit jamais devenir un tremplin. Mesures appliquees (NIST SP 800-190) : conteneurs **non-root**, `read_only`, `tmpfs` pour l'écriture temporaire, `cap_drop: ALL,no-new-privileges` et reseau bridge isole.

9.2 Conformite RGPD et article 323-1

Le systeme est **strictement passif** : il n'attaque ni ne sonde personne (le *hack-back* est prohibe par l'art. 323-1 du Code penal). Les adresses IP, donnees a caractere personnel, sont traitees au titre de l'**interet legitime** (securite du systeme d'information), avec anonymisation au /24 pour la diffusion et une conservation limitee (30 jours). L'approche est conforme aux recommandations ENISA (Proactive Detection of Security Incidents).

10. Cartographie MITRE ATT&CK et exports defensifs

Les comportements observes sont relies a la matrice ATT&CK : T1110 (Brute Force), T1595 (Active Scanning), T1190 (Exploit Public-Facing Application), T1059 (Command and Scripting Interpreter) et T1083 (File and Directory Discovery). A partir des sessions classees, le systeme genere automatiquement :

- des regles **Sigma** (une par profil, niveau high pour bruteforcer/bot), integrables a un SIEM ;
- un bundle **STIX 2.1** (indicateurs `ipv4-addr`, confiance = score AbuseIPDB), destine a MISP ;
- une liste de blocage **iptables** (`-A INPUT -s <IP> -j DROP`).

11. Gestion de projet et repartition du travail

Le projet a ete mene en binome sur 35 heures. La repartition indicative est la suivante :

primary!8Membre	Contributions principales
Mohammed Sellak	Services leures, pipeline d'analyse, tableau de bord, audit de furtivite
Rachid Oubenazha	Scenarios d'attaque, classification, conteneurisation, documentation

TABLE 6 – Repartition indicative des contributions (travail largement collaboratif).

12. Retour d'experience et perspectives

12.1 Bilan

La chaine *capture* → *analyse* → *CTI* est fonctionnelle, testee (5/5) et documentee. Le projet demontre la maitrise de Docker, du developpement Python securise, de l'analyse comportementale et du cadre legal.

12.2 Difficultes rencontrees

La persistance de la classification a necessite un correctif (validation transactionnelle SQLite : ajout du commit apres mise a jour du profil). La furtivite vis-a-vis de p0f reste la limite la plus difficile a lever sans intervenir sur le noyau.

12.3 Perspectives

En-tetes HTTP complets (ETag, Last-Modified), host-key SSH persistante, normalisation du TTL reseau, migration SQLite → PostgreSQL et alerting temps reel (MISP/SIEM). Objectif realiste : 25–26/30 en furtivite sans exposer de veritable systeme.

13. Conclusion

Nous avons concu, realise et valide un honeypot intelligent multi-services couvrant l'ensemble de la chaine, de la capture d'attaque jusqu'a la production de renseignement defensif. Les resultats (pres de 6 000 evenements captures, classification automatique, integrite parfaite, furtivite portee a 21/30) confirment la pertinence de l'approche medium-interaction et la solidite de l'implementation. Le systeme constitue une base reutilisable pour la detection proactive et l'entrainement d'une equipe defensive.

A. Annexes

A.1 Demarrage rapide

```
1 docker compose up -d --build          # 6 conteneurs
2 bash attacks/run_all.sh                # Hydra (SSH/FTP) + Nikto (HTTP)
3 python3 attacks/assertions.py          # 5/5 PASS attendu
4 bash audit/run_audit.sh                # sondes de detectabilite
5 python3 audit/score.py                  # score /30
6 # Tableau de bord : http://127.0.0.1:8050/
```

Listing 3 – Mise en route et validation.

A.2 Exemple de regle Sigma generee

```
1 title: HoneyPot - Brute force detecte
2 status: experimental
3 logsource:
4   product: honeypot
5 detection:
6   selection:
7     profile: bruteforcer
8     condition: selection
9 level: high
10 tags:
11   - attack.t1110
```

Listing 4 – Regle Sigma (profil bruteforcer).

A.3 Structure du depot

```
1 honeypots/    ssh | http | ftp    (services leurres)
2 common/      hp_common          (signature HMAC, contrat de log)
3 shipper/     transport des logs vers l'API
4 analyzer/    api, classifieur, storage, enrichers, exporters
```

```
5 dashboard/    app.py (Dash)
6 attacks/      run_all.sh, assertions.py
7 audit/        run_audit.sh, p0f_capture.sh, score.py
8 datasets/     listes (mots de passe, users, chemins, UA bots)
9 schemas/      event.schema.json (contrat de log)
10 tests/        tests unitaires
11 docs/         rapport.tex, presentation.html, script-oral.md, audits
12 docker-compose.yml, Makefile, pyproject.toml, README.md
```

Listing 5 – Arborescence simplifiée.